

CS360 Introduction to Database

The Database Language SQL

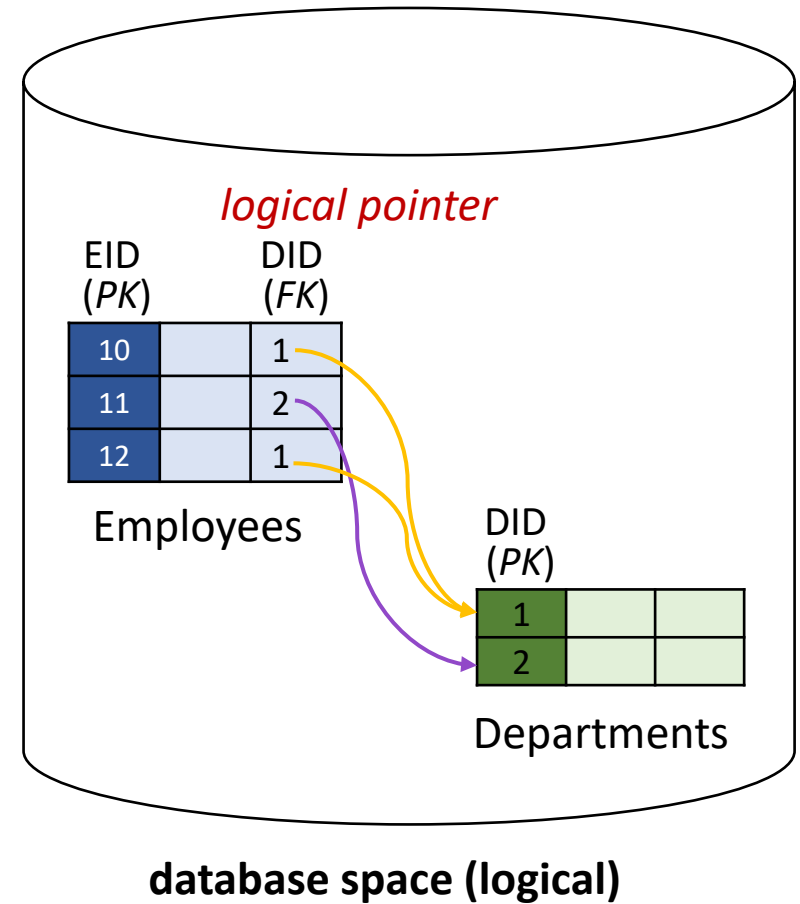
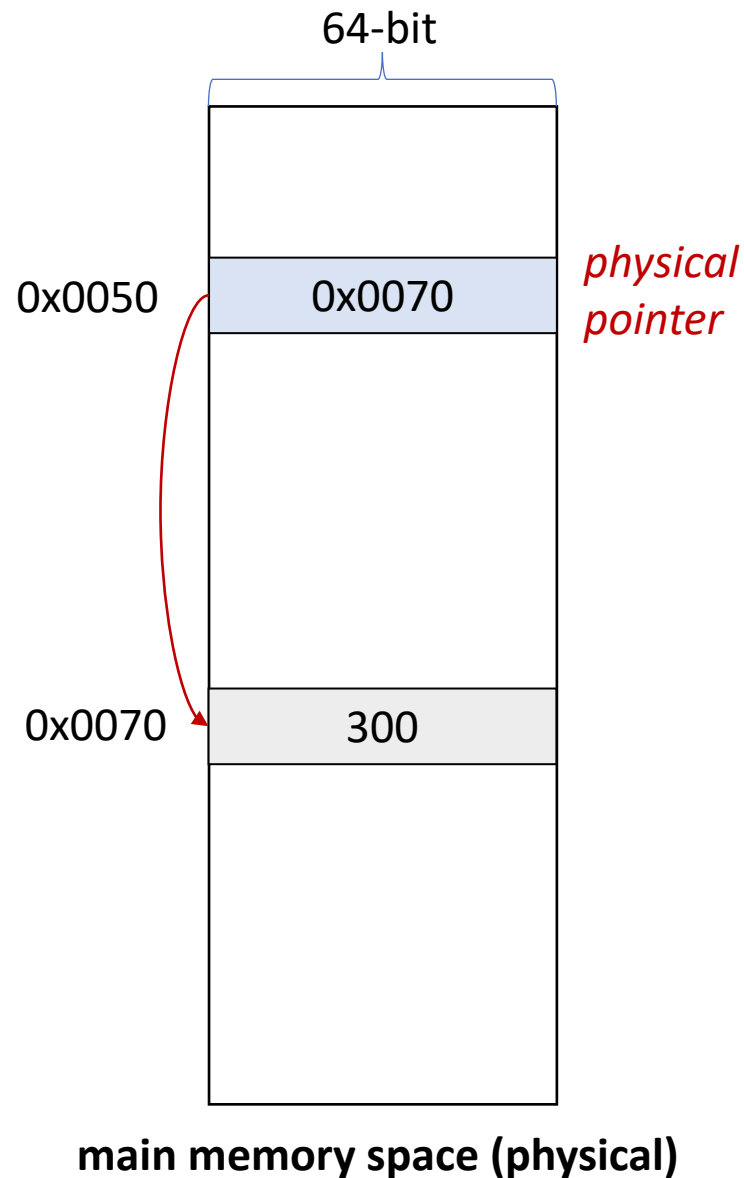
Min-Soo Kim

School of Computing, KAIST

Primary key, Foreign key,
Referential integrity,
Physical and logical pointer

Physical vs. Logical Pointer

EXTRA

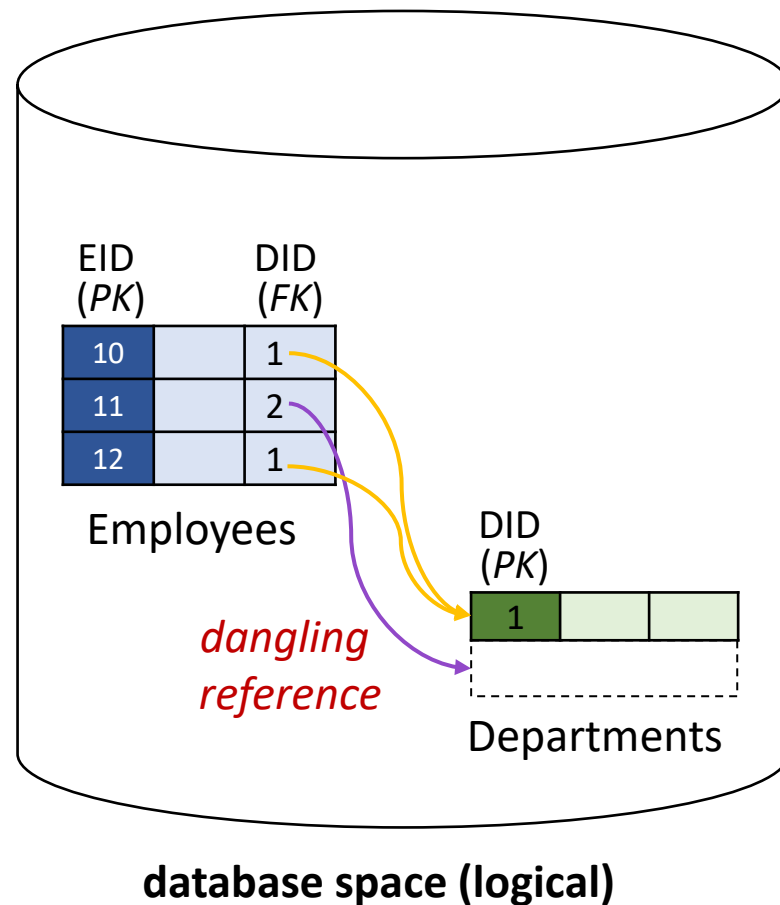
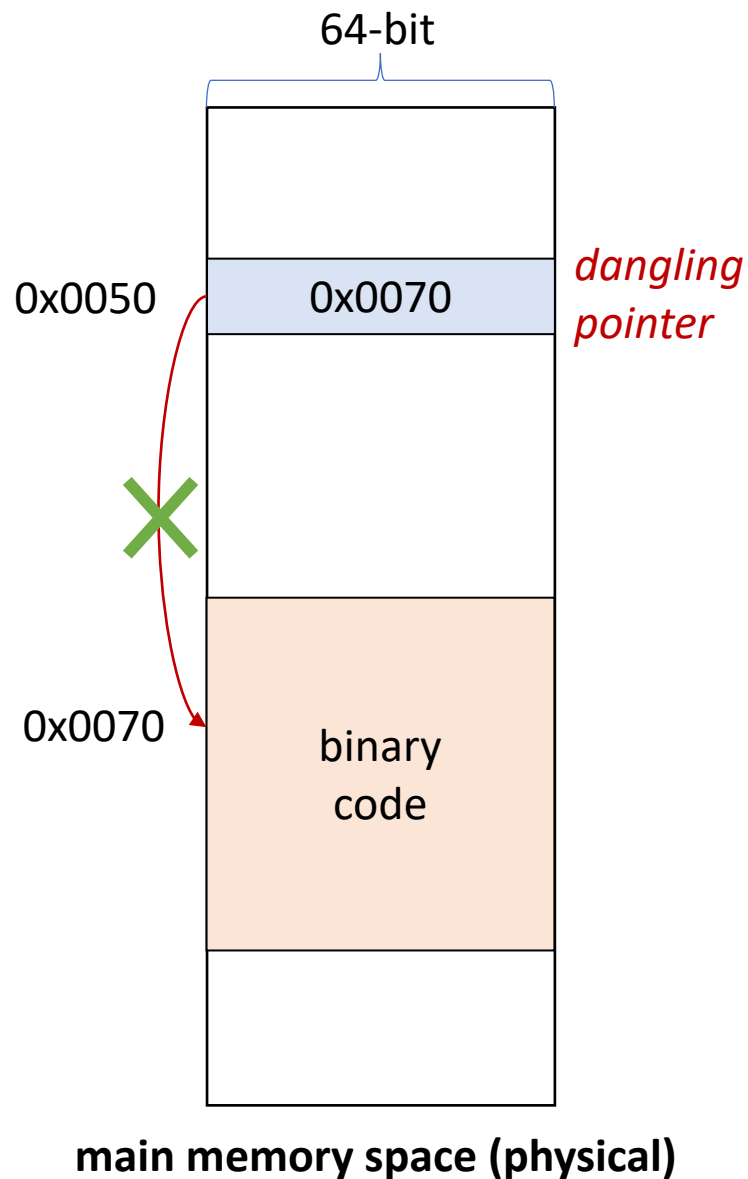


Foreign keys, referential integrity

- **Foreign key (FK)**: set of attributes in one relation that is used to `refer` to a tuple in another relation
 - FK must correspond to PK of the second relation
 - like a **logical pointer**
- e.g., **sid** is a foreign key referring to **Students(sid,)** :
 - **Enrolled**(sid: string, cid: string, grade: string)
 - **referential integrity** is achieved, if all foreign keys exist in the primary keys in another relation (i.e., **no dangling references**)
 - example of dangling references: broken links in HTML

Physical vs. Logical Pointer

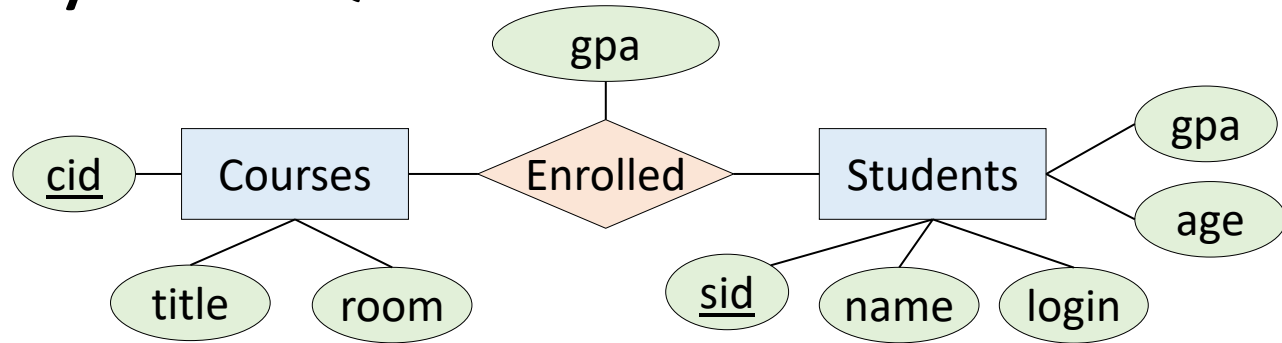
EXTRA



Foreign keys, referential integrity (*revisited*)

- **Foreign key (FK)**: set of attributes in one relation that is used to `refer` to a tuple in another relation
 - FK must correspond to PK of the second relation
 - like a **logical pointer**
- e.g., **sid** is a foreign key referring to **Students**:
 - **Enrolled**(sid: string, cid: string, grade: string)
 - **referential integrity** is achieved, if all foreign keys exist in the primary keys in another relation (i.e., **no dangling references**)
 - example of dangling references: broken links in HTML

Foreign keys in SQL



```

CREATE TABLE Enrolled
  (sid CHAR(20), cid CHAR(20), grade CHAR(2),
   PRIMARY KEY (sid,cid),
   FOREIGN KEY (sid) REFERENCES Students )
  
```

Enrolled

sid	cid	grade
53666	Carnatic101	C
53666	Reggae203	B
53650	Topology112	A
53666	History105	B

Students

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

Enforcing referential integrity

- Insertion

- if an Enrolled tuple with a non-existent student *id* tries to be inserted?
- DBMS's reject it!

- Deletion

- if a Students tuple is deleted?
- option1: disallow deletion of a Students tuple that is referred to
- option2: also delete all Enrolled tuples that refer to it
- option3: set *sid* in Enrolled tuples that refer to it to a default *sid* or null (denoting 'unknown' or 'inapplicable')

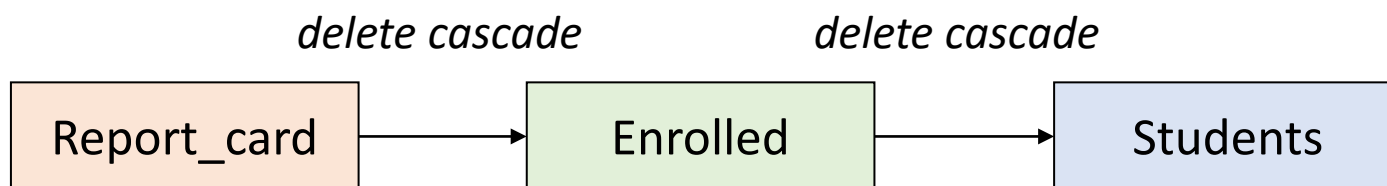
- Update

- if primary key of Students tuple is updated?
- DBMS's handle similarly to the deletion case

Referential integrity in SQL

- SQL92 and SQL99 support the following options on deletes and updates
 - ① Default is **NO ACTION** (delete/update is rejected)
 - ② **CASCADE** (also delete all tuples that refer to deleted tuple)
 - ③ **SET NULL / SET DEFAULT** (sets foreign key value of referencing tuple)

```
CREATE TABLE Enrolled
(sid CHAR(20),
cid CHAR(20),
grade CHAR(2),
PRIMARY KEY (sid,cid),
FOREIGN KEY (sid)
REFERENCES Students
ON DELETE CASCADE
ON UPDATE SET DEFAULT )
```



Simple SQL

Structured Query Language (SQL)

- Capability of SQL is very close to that of relational algebra
- SQL also includes statements for
 - modifying the database (e.g., inserting and deleting tuples)
 - declaring a database schema (i.e., data-definition language)
- Versions of SQL
 - ANSI (American National Standards Institute) SQL
 - SQL-92 (SQL2, SQL:1992): primary focus of this course
 - SQL-99 (SQL3, SQL:1999): supporting object-relational features
 - SQL:2023: supporting graph query processing features

Simple queries in SQL (SELECT-FROM-WHERE)

- **FROM:** the relation or relations to which the query refers
- **WHERE:** **selection-condition** in relational algebra
 - tuples must satisfy the condition in order to match the query
- **SELECT:** **projection-condition** in relational algebra
 - * indicates that the entire tuple is produced

`Movies(title, year, length, genre, studioName, producerC#)`

```
SELECT *  
FROM Movies  
WHERE studioName = 'Disney' AND year = 1990;
```

title	year	length	genre	studioName	producerC#
Pretty Woman	1990	119	romance	Disney	999

Projection in SQL

```
SELECT title, length
FROM Movies
WHERE studioName = 'Disney' AND year = 1990;
```

title	length
Pretty Woman	119

- We may follow the name of the attribute by AS and an alias, which becomes the header in the result relation

```
SELECT title AS name, length AS duration
FROM Movies
WHERE studioName = 'Disney' AND year = 1990;
```

name	duration
Pretty Woman	119

```
SELECT title AS name, length*0.016667 AS lengthInHours
FROM Movies
WHERE studioName = 'Disney' AND year = 1990;
```

name	lengthInHours
Pretty Woman	1.98334

- SELECT allows a constant as an expression
 - for applications to put some useful words into the output

```
SELECT title, length*0.016667 AS length, 'hrs.' AS inHours
FROM Movies
WHERE studioName = 'Disney' AND year = 1990;
```

title	length	inHours
Pretty Woman	1.98334	hrs.

Selection in SQL

- WHERE includes conditional expressions like those in common languages (e.g., C or Java)
 - **comparison operators**: =, <>, <, >, <=, >=
 - values of constants and attributes may be compared
 - e.g., studioName = 'Disney'
 - result of a comparison is a Boolean value: TRUE or FALSE
 - **arithmetic operators**: +, *, ...
 - **string operators**: || (for concatenation), ...
 - e.g., 'foo' || 'bar' has value 'foobar'
 - **logical operators** for Boolean values: AND, OR, NOT

```
SELECT title
FROM Movies
WHERE (year > 1970 OR length < 90) AND studioName = 'MGM';
```

Comparison of strings

- Comparing strings by “less than” operators (e.g., $<$ or $\geq$)
 - ask whether one precedes the other in **lexicographic order**
 - i.e., in dictionary order, or alphabetically
- We say $a_1 a_2 \dots a_n < b_1 b_2 \dots b_m$
 - if either $a_1 < b_1$, or
 - if $a_1 = b_1$ and $a_2 < b_2$, or
 - if $a_1 = b_1$, $a_2 = b_2$ and $a_3 < b_3$, and so on
 - e.g., ‘fodder’ $<$ ‘foo’
- We also say $a_1 a_2 \dots a_n < b_1 b_2 \dots b_m$
 - if $n < m$ and $a_1 a_2 \dots a_n = b_1 b_2 \dots b_n$
 - e.g., ‘bar’ $<$ ‘bargain’

Pattern matching in SQL

- Expression ***s* LIKE *p***: true *if and only if* *s* matches *p*
 - *s*: a string
 - *p*: a pattern
- Two special characters
 - %: can match any sequence of 0 or more characters in *s*
 - _: can match any one character in *s*

```
SELECT title
FROM Movies
WHERE title LIKE 'Star _____';
```

title
Star Wars
Star Trek

```
SELECT title
FROM Movies
WHERE title LIKE '%'' 's%';
```

title
Logan's Run
Alice's Restaurant

two consecutive apostrophes represent a single apostrophe in *s*

Dates and times

- Date constant: keyword DATE + a string of a special form
 - e.g., DATE '1948-05-14'
 - one-digit month or day should be padded with a leading 0
- Time constant: keyword TIME + a string of a special form
 - e.g., TIME '15:00:02'
 - two digits for the hour in 24-hour
- Timestamp constant: keyword TIMESTAMP + date + time
 - we use a timestamp value to combine dates and times
 - e.g., TIMESTAMP '1948-05-14 12:00:00'
- Comparison operators for dates or times
 - $date1 < date2$: the first date is earlier than the second date

Null values

- Many different interpretations about null values
 - ① value **unknown**: “I know there is some value, but I don’t know what it is.”
 - e.g., an unknown birthdate
 - ② value **inapplicable**: “There is no value that makes sense here.”
 - e.g., the *spouse* attribute in *MovieStar* for an unmarried star
 - ③ value **withheld**: “We are not entitled to know the value.”
(We have no right to know the value)
 - e.g., an unlisted phone number might be NULL in the *phone* attribute

Operating upon NULL values

- Two rules
 1. result of an arithmetic operator (e.g., x or $+$) is **NULL**
 - when operating on a NULL and any value, including another NULL
 2. result of a comparison operator (e.g., $=$ or $>$) is **UNKNOWN**
 - when comparing a NULL value and any value, including another NULL
- Note: **NULL is not a constant**
 - we cannot use NULL explicitly as an operand
- e.g., when a variable x has the NULL value
 - value of $x + 3$: NULL
 - $NULL + 3$: not a legal SQL expression
 - value of $x = 3$: UNKNOWN
 - we cannot tell if the value of x equals the value 3
 - comparison $NULL = 3$: not correct SQL

Three-valued logic

- Result of logical operators AND, OR, and NOT: truth-values
- Truth-values: {TRUE, FALSE, UNKNOWN}

x	y	x AND y	x OR y	NOT x
TRUE	TRUE	TRUE	TRUE	FALSE
TRUE	UNKNOWN	UNKNOWN	TRUE	FALSE
TRUE	FALSE	FALSE	TRUE	FALSE
UNKNOWN	TRUE	UNKNOWN	TRUE	UNKNOWN
UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN
UNKNOWN	FALSE	FALSE	UNKNOWN	UNKNOWN
FALSE	TRUE	FALSE	TRUE	TRUE
FALSE	UNKNOWN	FALSE	UNKNOWN	TRUE
FALSE	FALSE	FALSE	FALSE	TRUE

truth table for three-valued logic

Example

- Suppose there are *Movies* tuples with NULL in *length*
 - comparison ($\text{length} \leq 120$) evaluates to UNKNOWN
 - comparison ($\text{length} > 120$) evaluates to UNKNOWN
 - OR of two UNKNOWN's is UNKNOWN
 - WHERE clause evaluates to UNKNOWN
 - such a tuple is not returned as part of the answer to the query
- True meaning of the query
 - “find all the *Movies* tuples with non-NULL lengths.”

```
SELECT *  
FROM Movies  
WHERE length <= 120 OR length > 120;
```

Ordering the output

- Tuples produced by a query are presented in sorted order
- Syntax: **ORDER BY <list of attributes>**
 - the order is by default ascending (or by the keyword ASC)
 - the order becomes descending by the keyword DESC
 - ordering is performed on the result of FROM, WHERE, and other clauses, just before applying SELECT

• e.g.,

```
SELECT *  
FROM Movies  
WHERE studioName = 'Disney' AND year = 1990  
ORDER BY length, title;
```

• e.g.,

```
SELECT *  
FROM R  
ORDER BY A+B DESC;
```

for a relation R(A,B)

Complex SQL

Products and joins in SQL

- Query: the names of the producers of Star Wars
- Input relations required
 - Movies(title, year, length, genre, studioName, producerC#)
 - MovieExec(name, address, cert#, netWorth)
- SQL query

```
SELECT name
FROM Movies, MovieExec
WHERE title = 'Star Wars' AND producerC# = cert#;
```

- Answer

name
George Lucas

Example database schema about movies (revisited)

Movies (

title: string,
year: integer,
length: integer,
genre: string,
studioName: string,
producerC#: integer

)

MovieStar (

name: string,
address: string,
gender: char,
birthdate: date

)

StarsIn (

movieTitle: string,
movieYear: integer,
starName: string

)

MovieExec (

name: string
address: string,
cert#: integer,
netWorth: integer

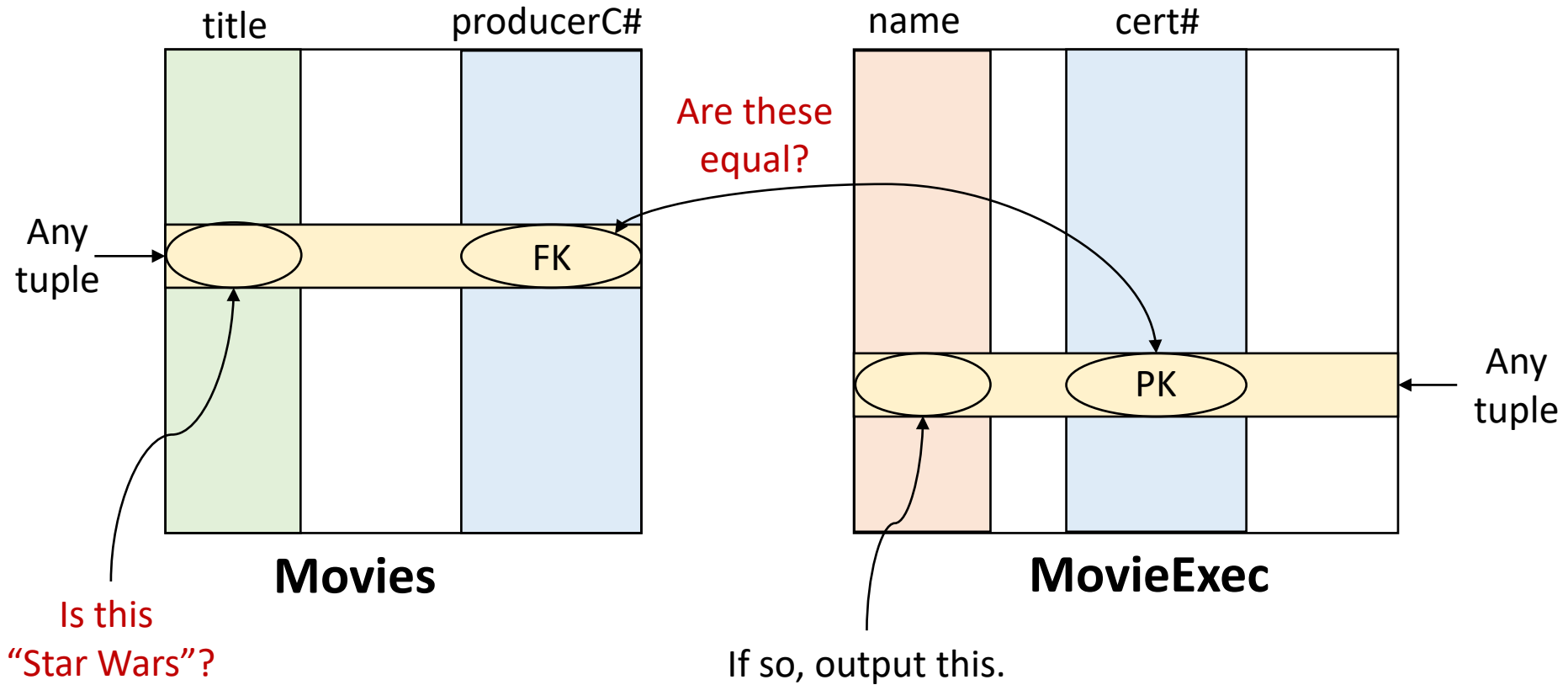
)

Studio (

name: string,
address: string,
presC#: integer

)

Pair every tuple of Movies with every tuple of MovieExec
and **test two conditions**



Disambiguating attributes

- Among several relations involved in a query
 - two or more attributes may have the same name
 - SQL solves this issue by using *<relation name>.<attribute name>*

- e.g.,

```
MovieStar(name, address, gender, birthdate)
MovieExec(name, address, cert#, netWorth)
```

```
SELECT MovieStar.name, MovieExec.name
FROM MovieStar, MovieExec
WHERE MovieStar.address = MovieExec.address;
```

MovieStar.name	MovieExec.name
Jane Fonda	Ted Turner

- c.f.,

```
SELECT MovieStar.name
FROM Movies, MovieExec
WHERE Movie.title = 'Star Wars'
      AND Movie.producerC# = MovieExec.cert#
```

Tuple variables

- Sometimes we need to ask a query that involves two or more tuples from the same relation
 - e.g., one contract involving two studios in Chap. 4
 - e.g., query for finding paths from the *Edge* relation
- We list a relation R as many times as we need in FROM
 - use **alias** (**tuple variable**) for each occurrence of R in FROM
- e.g., two tuple variables, Star1 and Star2

```
SELECT Star1.name, Star2.name
FROM MovieStar Star1, MovieStar Star2
WHERE Star1.address = Star2.address
      AND Star1.name < Star2.name;
```

Star1.name	Star2.name
Paul Newman	Joanne Woodward
Joanne Woodward	Paul Newman

Union, intersection, and difference (1)

- e.g., “the names and addresses of all female movie stars who are also movie executives with a net worth over \$10,000,000.”

```
MovieStar(name, address, gender, birthdate)
MovieExec(name, address, cert#, netWorth)
```

```
(SELECT name, address
 FROM MovieStar
 WHERE gender = 'F')
INTERSECT
(SELECT name, address
 FROM MovieExec
 WHERE netWorth > 10000000);
```

Union, intersection, and difference (2)

- e.g., “the names and addresses of movie stars who are not also movie executives, regardless of gender or net worth.”

```
(SELECT name, address FROM MovieStar)  
EXCEPT  
(SELECT name, address FROM MovieExec)
```

- e.g., “all the titles and years of movies that appeared in either the *Movies* or *StarsIn* relation.”

```
Movies(title, year, length, genre, studioName, producerC#)  
StarsIn(movieTitle, movieYear, starName)
```

```
(SELECT title, year FROM Movie)  
UNION  
(SELECT movieTitle AS title, movieYear AS year FROM StarsIn);
```

Subqueries

- Subquery: a query that is part of another
 - surrounded by parentheses
- Subqueries can also have subqueries
 - down as many levels as we wish
- Subqueries are used in union, intersection, difference
 - for connecting two subqueries to form the whole query
- Other ways that subqueries can be used:
 - subqueries can return a single constant
 - the constant can be compared with another value in **WHERE**
 - subqueries can return relations
 - the relation can be used in various ways in **WHERE**
 - subqueries can appear in **FROM**
 - followed by a tuple variable that represents the tuples in the result of the subquery

Subqueries that produce scalar values

- Return a single constant
 - used to compare its result to a constant or attribute in WHERE

```
Movies(title, year, length, genre, studioName, producerC#)  
MovieExec(name, address, cert#, netWorth)
```

```
SELECT name  
FROM Movies, MovieExec  
WHERE title = 'Star Wars' AND producerC# = cert#;
```

```
SELECT name  
FROM MovieExec  
WHERE cert# =  
    (SELECT producerC#  
     FROM Movies  
     WHERE title = 'Star Wars')  
    ;
```



```
SELECT name  
FROM MovieExec  
WHERE cert# = 12345;
```

Conditions involving relations

- Subqueries **return a one-column relation**
 - SQL operators such as **EXISTS, IN, NOT IN, > ALL, > ANY**
 - they **take a relation R (subquery)** and produce a Boolean result
- **EXISTS R**
 - a condition that is true *if and only if* R is **not empty**
- **s IN R**
 - true *if and only if* s is equal to **one of the values** in R
- **s NOT IN R**
 - true *if and only if* s is equal to **no value** in R

- $s > \text{ALL } R$
 - true *iff* s is greater than every value in R
 - $>$ operator could be replaced by any of comparison operators
 - e.g., $s <> \text{ALL } R$: the same as $s \text{ NOT IN } R$
- $s > \text{ANY } R$
 - true *iff* s is greater than at least one value in R
 - $>$ operator could be replaced by any of comparison operators
 - e.g., $s = \text{ANY } R$: the same as $s \text{ IN } R$
- EXISTS, ALL, and ANY can be negated by putting NOT
 - e.g., NOT EXISTS R : true *iff* R is empty
 - e.g., NOT $s \geq \text{ALL } R$: true *iff* s is not the maximum value in R
 - e.g., NOT $s > \text{ANY } R$: true *iff* s is the minimum value in R

Conditions involving tuples

- Tuple t has the same number of attributes as a relation R ,
 - then, it makes sense to compare t and R as in the previous slide

```
Movies(title, year, length, genre, studioName, producerC#)
StarsIn(movieTitle, movieYear, starName)
MovieExec(name, address, cert#, netWorth)
```

```
SELECT name
FROM MovieExec
WHERE cert# IN
    (SELECT producerC#
     FROM Movies
     WHERE (title, year) IN
         (SELECT movieTitle, movieYear
          FROM StarsIn
          WHERE starName = 'Harrison Ford'
         )
    )
;
```

return the set of producers of
Harrison Ford's movies

we should analyze any
query with subqueries
from the inside out

title	year
Star Wars	1977
Raiders of the Lost Ark	1981
The Fugitive	1993
...	...

- Many nested queries can be written as a single expression
 - with relations in the FROM clause
 - for each of the relations mentioned in the main or subqueries
- **IN** relationships are replaced by **equalities** in WHERE

```
SELECT name
FROM MovieExec
WHERE cert# IN
  (SELECT producerC#
   FROM Movies
   WHERE (title, year) IN
     (SELECT movieTitle, movieYear
      FROM StarsIn
      WHERE starName = 'Harrison Ford'
     )
  )
;
```

```
SELECT name
FROM MovieExec, Movies, StarsIn
WHERE cert# = producerC# AND
      title = movieTitle AND
      year = movieYear AND
      starName = 'Harrison Ford';
```

Correlated subqueries

- Subqueries so far are **simple**
 - can be **evaluated once**; their result **can be used in a higher level**
- **Correlated subquery** (more complicated subqueries)
 - require to be **evaluated many times**
 - once for each assignment of a value to some term in the subquery that comes from a tuple variable outside the subquery

```
SELECT title
FROM Movies Old
WHERE year < ANY
    (SELECT year
     FROM Movies
     WHERE title = Old.title
    );
```

the subquery whether there is a movie
with the same title and a greater year

replaced by a constant string such as
'King Kong' for a value of Old.title of
each tuple of Movies

Scoping rules

- When writing a correlated query
 - it is important to aware of the **scoping rules for names**

```
SELECT title
FROM Movies Old
WHERE year < ANY
    (SELECT year
     FROM Movies
     WHERE title = Old.title
    );
```

Subqueries in FROM clauses

- We may use a subquery instead of a stored relation in FROM
 - since we don't have a name for the result of the subquery
 - we **must give the subquery a tuple-variable alias**

```
SELECT name
FROM MovieExec, (SELECT producerC#
                  FROM Movies, StarsIn
                  WHERE title = movieTitle AND
                        year = movieYear AND
                        starName = 'Harrison Ford'
                  ) Prod
WHERE cert# = Prod.producerC#;
```


SQL join expression

- Cartesian product (just product): R **CROSS JOIN** S
- Theta-join: R **JOIN** S **ON** $\langle \text{condition} \rangle$
 - the product $R \times S$ is followed by a selection of the condition

```
Movies(title, year, length, genre, studioName, producerC#)
StarsIn(movieTitle, movieYear, starName)
```

```
Movies CROSS JOIN StarsIn;
```

```
Movies JOIN StarsIn ON
    title = movieTitle AND year = movieYear;
```

```
SELECT title, year, length, genre, studioName,
       producerC#, starName
FROM Movies JOIN StarsIn ON
    title = movieTitle AND year = movieYear;
```

the result is
a nine-column
relation
(two columns
are redundant)

Natural joins

- Natural join differs from theta-join
 - ① **join condition**: all pairs of attributes from two relations having a common name are equated (there are no other conditions)
 - ② **one of each pair of equated attributes is projected out**
- Form: $R \text{ NATURAL JOIN } S$

```
MovieStar(name, address, gender, birthdate)  
MovieExec(name, address, cert#, netWorth)
```

MovieStar **NATURAL JOIN** MovieExec;

Outer joins

- Augmenting the result of a join by the dangling tuples
 - padded with null values

```
MovieStar(name, address, gender, birthdate)
MovieExec(name, address, cert#, netWorth)
```

```
MovieStar NATURAL FULL OUTER JOIN MovieExec;
```

name	address	gender	birthdate	cert#	networth
Mary Tyler Moore	Maple St.	'F'	9/9/99	12345	\$100...
Tom Hanks	Cherry Ln.	'M'	8/8/88	NULL	NULL
George Lucas	Oak Rd.	NULL	NULL	23456	\$200...

```
MovieStar NATURAL LEFT OUTER JOIN MovieExec;
```

```
MovieStar NATURAL RIGHT OUTER JOIN MovieExec;
```

Theta-outer join

- Form: R **FULL OUTER JOIN** S **ON** $\langle \text{condition} \rangle$

```
Movies(title, year, length, genre, studioName, producerC#)
StarsIn(movieTitle, movieYear, starName)
```

```
Movies FULL OUTER JOIN StarsIn ON
    title = movieTitle AND year = movieYear;
```

```
Movies LEFT OUTER JOIN StarsIn ON
    title = movieTitle AND year = movieYear;
```

```
Movies RIGHT OUTER JOIN StarsIn ON
    title = movieTitle AND year = movieYear;
```

Eliminating duplicates

- SQL system does **not ordinarily eliminate duplicates**
 - results of SQL queries may list the same tuple several times
 - c.f., SQL system does not ordinarily order the result tuples
- Form: **SELECT DISTINCT**

```
SELECT DISTINCT name
FROM MovieExec, Movies, StarsIn
WHERE cert# = producerC# AND
       title = movieTitle AND
       year = movieYear AND
       starName = 'Harrison Ford';
```

Duplicates in unions, intersections, and differences

- Union, intersection, difference **normally eliminate duplicates**
 - bags are converted to sets
 - the set version of the operation is applied
- In order to prevent the elimination of duplicates
 - $R \text{ UNION } \textcolor{red}{ALL} S$
 - the number of a tuple t appears: that in R + that in S
 - $R \text{ INTERSECT } \textcolor{red}{ALL} S$
 - the number of a tuple t appears: the minimum of that in R and that in S
 - $R \text{ EXCEPT } \textcolor{red}{ALL} S$
 - the number of a tuple t appears: that in R - that in S

```
(SELECT title, year FROM Movies)
```

```
UNION ALL
```

```
(SELECT movieTitle AS title, movieYear AS year FROM StarsIn);
```

Aggregation operators

- SUM, AVG, MIN, MAX, COUNT, ...
- COUNT(*)
 - counts all the tuples in the relation from FROM and WHERE
- COUNT(DISTINCT x)
 - counts the number of distinct values in column x
 - we can use any of the other operators in place of COUNT
- AVG(x): $\text{SUM}(x) / \text{COUNT}(x)$

```
SELECT COUNT (*)  
FROM StarsIn;
```

```
SELECT COUNT (starName)  
FROM StarsIn;
```

```
SELECT COUNT (DISTINCT starName)  
FROM StarsIn;
```

```
SELECT AVG (netWorth)  
FROM MovieExec;
```

Grouping

- GROUP BY: followed by a list of *grouping* attributes
 - the tuples are grouped according to their values in the grouping attributes
- Aggregation operators in SELECT: applied only within groups
 - only attributes in GROUP BY can be used in SELECT, except aggregated attributes
- e.g., **sum of the lengths** of all movies for **each studio**

```
Movies(title, year, length, genre, studioName, producerC#)
```

```
SELECT studioName, SUM(length)
FROM Movies
GROUP BY studioName;
```



```
SELECT studioName  
FROM Movies  
GROUP BY studioName;
```

=
same
effect

```
SELECT DISTINCT studioName  
FROM Movies;
```

- GROUP BY for multiple relations is interpreted as
 - evaluate the resulting relation R expressed by FROM and WHERE
 - group the tuples of R according to the attributes in GROUP BY
 - produce the final result of attributes and aggregations in SELECT

```
Movies(title, year, length, genre, studioName, producerC#)  
MovieExec(name, address, cert#, netWorth)
```

```
SELECT name, SUM(length)  
FROM MovieExec, Movies  
WHERE producerC# = cert#  
GROUP BY name;
```

Grouping, aggregation, and nulls

- NULL value is ignored in any aggregation
 - not contribute to sum, average, count, minimum, or maximum
 - e.g., COUNT(*): always counts the number of tuples in a relation
 - e.g., COUNT(X): counts the number of non-NULL values of X
- NULL is treated as an ordinary value when forming groups
 - i.e., we can have a group in which grouping attributes are NULL
- Result of any aggregation over an empty bag is NULL
 - except COUNT (result of COUNT over an empty bag is 0)

Example

A	B
NULL	NULL

```
SELECT A, COUNT(B)
FROM R
GROUP BY A;
```



A	COUNT(B)
NULL	0

```
SELECT A, SUM(B)
FROM R
GROUP BY A;
```



A	SUM(B)
NULL	NULL

HAVING clauses

- GROUP BY followed by HAVING <condition>
 - restrict the groups to the ones satisfying <condition>
- e.g.,
 - the total film length for only the producers who made at least one film prior to 1930

```
Movies(title, year, length, genre, studioName, producerC#)  
MovieExec(name, address, cert#, netWorth)
```

```
SELECT name, SUM(length)  
FROM Movies, MovieExec  
WHERE producerC# = cert#  
GROUP BY name  
HAVING MIN(year) < 1930;
```

name	MIN(year)	SUM(length)
George Lucas	1973	5012
Lewis Milestone	1929	240
...	...	...

DML

Database Manipulation Language (DML)

- Still use SQL, but not the form of SELECT-FROM-WHERE
- Not return a result, but rather change the state of DB
 - insert tuples into a relation
 - delete certain tuples from a relation
 - update values of certain attributes of certain existing tuples

Database modifications: insertion

- `INSERT INTO R(A1,...,An) VALUES (v1,...,vn);`
 - if the list of attributes does not include all attributes of R,
 - the tuple created has default values for all missing attributes
- e.g.,

```
INSERT INTO StarsIn(movieTitle, movieYear, starName)
VALUES('The Maltese Falcon', 1942, 'Sydney Greenstreet');
```

```
INSERT INTO StarsIn
VALUES('The Maltese Falcon', 1942, 'Sydney Greenstreet');
```

Example

- Suppose we want to add to the relation

`Studio(name, address, presC#)`

all movie studios that are mentioned in the relation

`Movies(title, year, length, genre, studioName, producerC#)`

but do not appear in Studio.

```
INSERT INTO Studio(name)
  SELECT DISTINCT studioName
  FROM Movies
  WHERE studioName NOT IN
    (SELECT name
     FROM Studio);
```


Database modifications: deletion

- `DELETE FROM R WHERE <condition>;`
 - every tuple satisfying the condition will be deleted from R
 - we cannot simply specify a tuple to be deleted
- e.g., `StarsIn(movieTitle, movieYear, starName)`

```
DELETE FROM StarsIn
WHERE movieTitle = 'The Maltese Falcon' AND
      movieYear = 1942 AND
      starName = 'Sydney Greenstreet';
```

- e.g., `MovieExec(name, address, cert#, netWorth)`

```
DELETE FROM MovieExec
WHERE netWorth < 10000000;
```

delete all movie executives whose
net worth is low (less than ten
million dollars)

Database modifications: updates

- **UPDATE R SET <new-value assignments> WHERE <condition>;**
 - each new-value assignment: **<an attribute> = <an expression>**
 - if more than one assignment, they are separated by commas
- **Effect**
 - find all the tuples in R that satisfy the condition
 - each of them is changed by the expressions in the assignments
- e.g., modify MovieExec by **attaching the title “Pres.” in front of the name** of **every movie executive who is the president of a studio**

```
MovieExec(name, address, cert#, netWorth)
```

```
UPDATE MovieExec
SET name = 'Pres. ' || name
WHERE cert# IN (SELECT presC# FROM Studio);
```

Thank you

Q & A